



Name: Muhammad Usama Bilal

Roll Number :CSI-B1-460

INSTRUCTOR: MUHAMMAD SAAD

Date: 03-5-2026

Table of Contents

- Engagement Overview
- Testing Strategy & Workflow
- Lab Environment & Configuration
- Reflected XSS Vulnerability Assessment
- Persistent (Stored) XSS Vulnerability Assessment
- Client-Side Attacks & Cookie Exploitation
- Cross-Site Request Forgery (CSRF) Evaluation
- Security Control Effectiveness (Low vs Medium vs High)
- Consolidated Vulnerability Matrix
- Mitigation & Hardening Recommendations
- Appendix – Payload Reference Library

1. Engagement Overview

This assessment focused on evaluating **client-side web application security weaknesses** using Damn Vulnerable Web Application (DVWA). The objective was to identify, exploit, and analyze vulnerabilities related to **Cross-Site Scripting (XSS)**, **Cross-Site Request Forgery (CSRF)**, and **cookie-based session exploitation** across multiple security configurations.

Testing was conducted under **Low, Medium, and High security levels** to measure the effectiveness of implemented defensive controls and to understand real-world attack scenarios.

High-Level Risk Summary

Severity	Issue Identified	Affected Levels
Critical	Arbitrary JavaScript execution via XSS	Low, Medium
Critical	Persistent Stored XSS affecting all users	Low, Medium
High	Session hijacking via cookie theft	All Levels
High	CSRF-based password manipulation	Low, Medium
Medium	Weak input filtering bypass	Medium
Low	Insecure cookie attribute configuration	All Levels

2. Testing Strategy & Workflow

- The assessment followed a structured purple-team methodology:
- Deployment and validation of vulnerable test environment
 - Client-side reconnaissance using browser developer tools

3. Reflected XSS discovery and payload execution
4. Stored XSS injection and persistence validation
5. Cookie theft and client-side data exfiltration
6. CSRF exploitation through forged requests
7. Defensive mechanism evaluation and bypass testing

Toolset Utilized

- **Browser:** Mozilla Firefox (DevTools – Inspector, Console, Network, Storage)
- **Web Server:** Python HTTP Server
- **Target Application:** DVWA (v1.0.7)
- **Operating System:** Kali Linux
- **Auxiliary Tools:** Nano (HTML editing), Burp Suite (traffic inspection)

3. Lab Environment & Configuration

3.1 Target Deployment

DVWA was installed and accessed locally via:

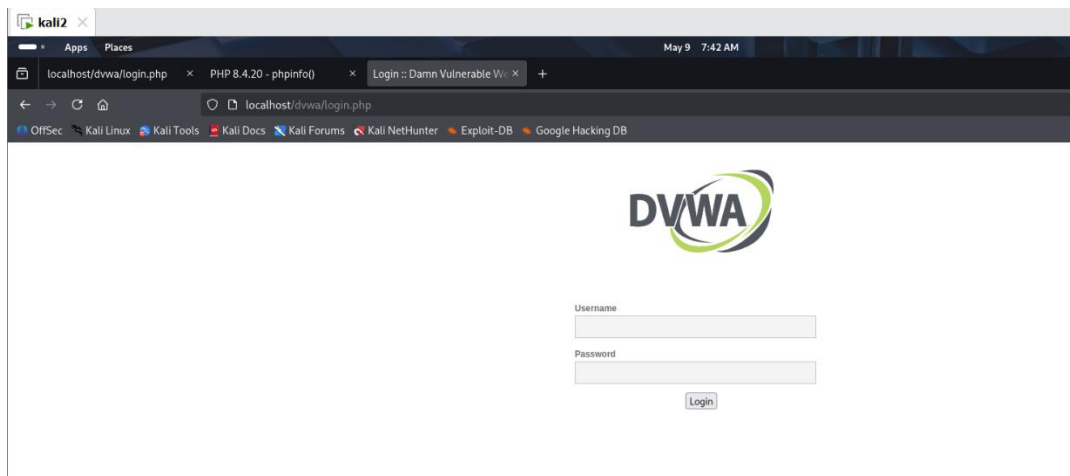
`http://192.168.0.108`

Default Credentials Used

- Username: **admin**
- Password: **password**

3.2 Security Level Setup

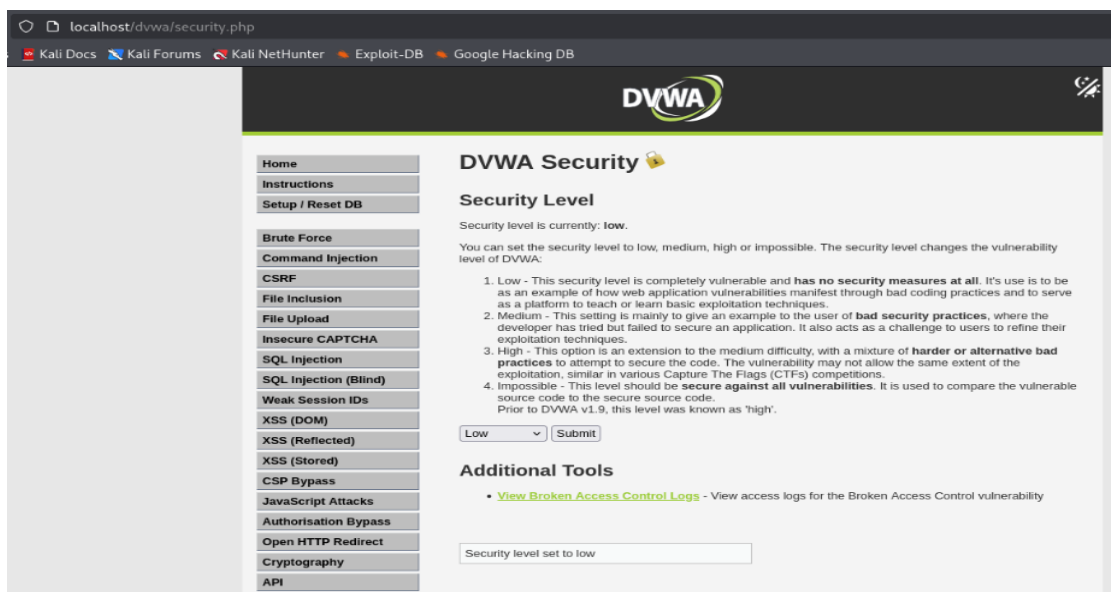
Testing was initially performed with **Low security**, followed by **Medium** and **High** levels to observe security control behavior.



3.3 Initial Cookie Analysis (Low Security)

Cookies observed using browser storage inspection:

Cookie Name	HttpOnly	Secure	SameSite
PHPSESSID	false	false	None
security	false	false	None



Security Implication

- Cookies accessible via JavaScript
- No HTTPS enforcement
- Cookies sent during cross-site requests

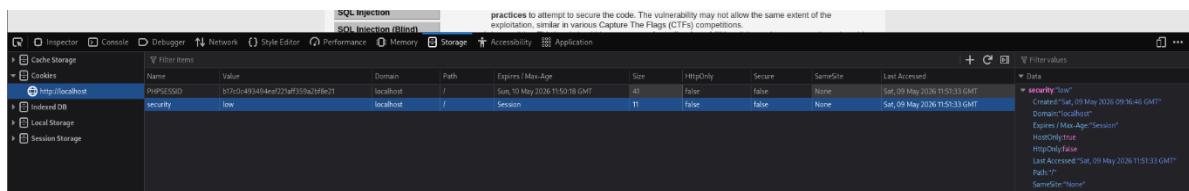
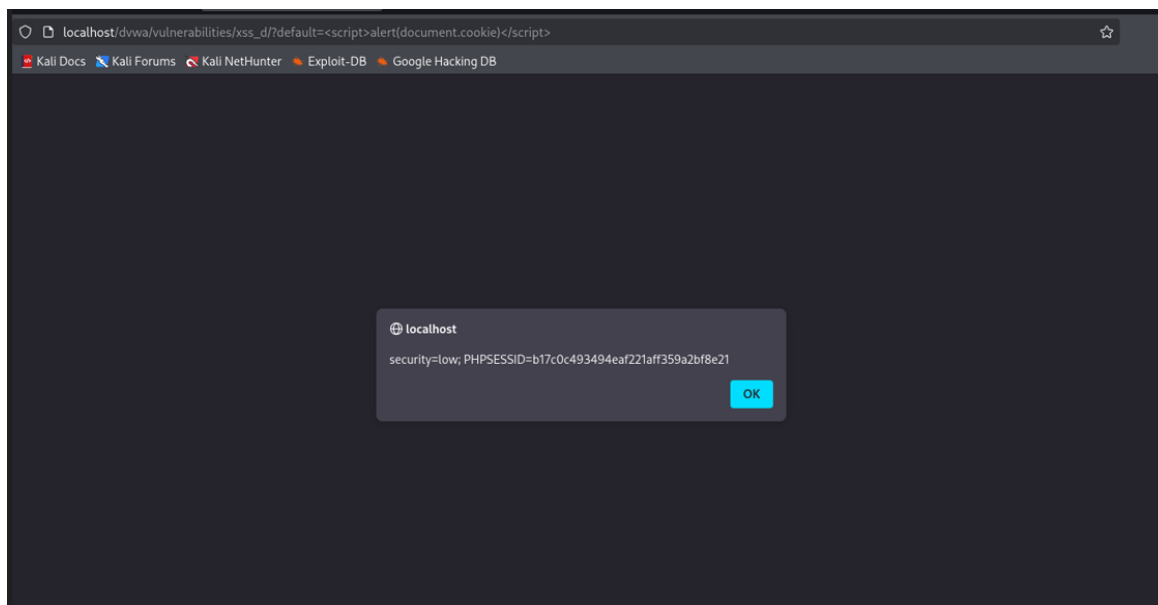
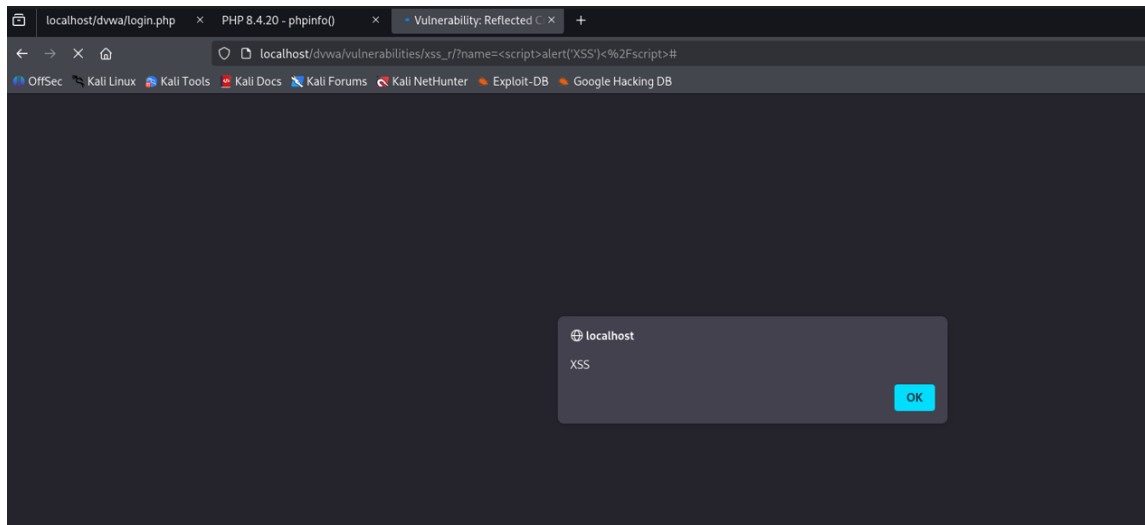
4. Reflected XSS Vulnerability Assessment

4.1 Description

Reflected XSS vulnerabilities were identified where user-controlled input was returned immediately in server responses without sanitization, allowing injected JavaScript to execute within the browser context.

4.2 Payload Testing (Low Security)

Payload	Outcome
<code><script>alert('XSS')</script></code>	Executed
<code><script>alert(document.cookie)</script></code>	Cookie exposed
UI defacement script	Page modified



4.3 Impact Evaluation

Security Area	Impact
Confidentiality	High
Integrity	High
Availability	Medium
Overall Risk	Critical

5. Persistent (Stored) XSS Vulnerability Assessment

5.1 Description

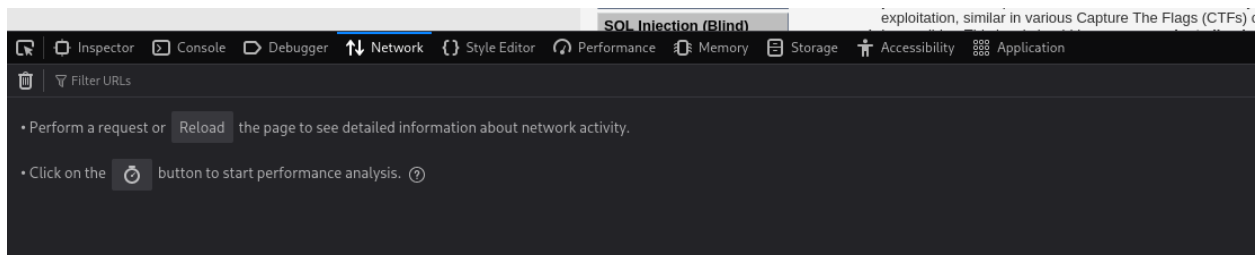
Stored XSS vulnerabilities were discovered where malicious input was permanently saved within the application database and executed for every user accessing the affected page.

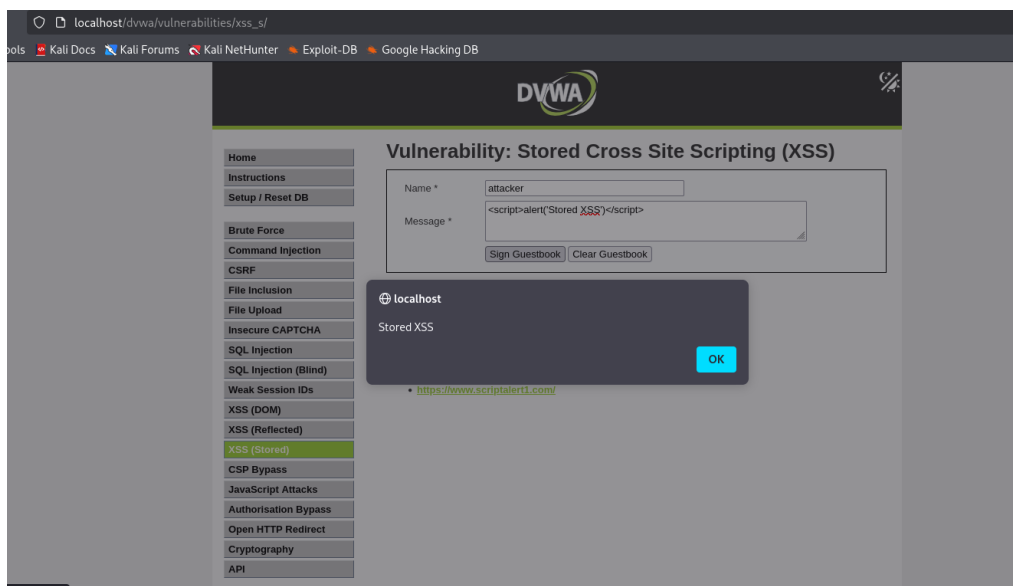
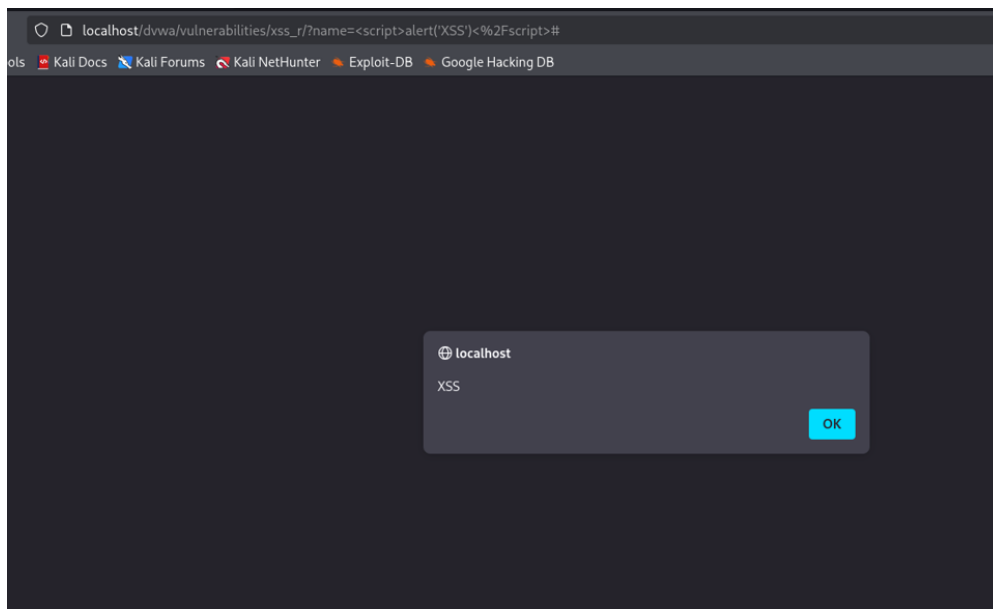
5.2 Injection Vector

Guestbook input fields on the XSS (Stored) module were used for payload insertion.

5.3 Payload Execution

```
<script>alert('Stored XSS')</script>
```





5.4 Persistence Verification

Action	Result
Initial submission	Executed
Page reload	Executed
New session	Executed

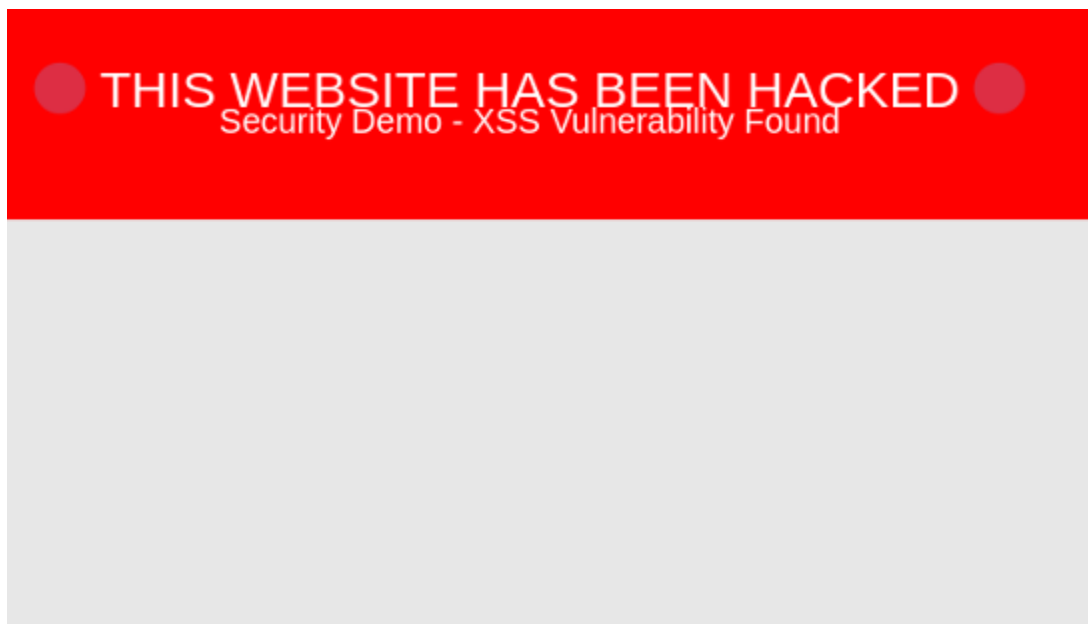
5.5 Risk Assessment

Area	Impact
Confidentiality	Critical
Integrity	Critical
Availability	High
Overall Severity	Critical

6. Client-Side Attacks & Cookie Exploitation

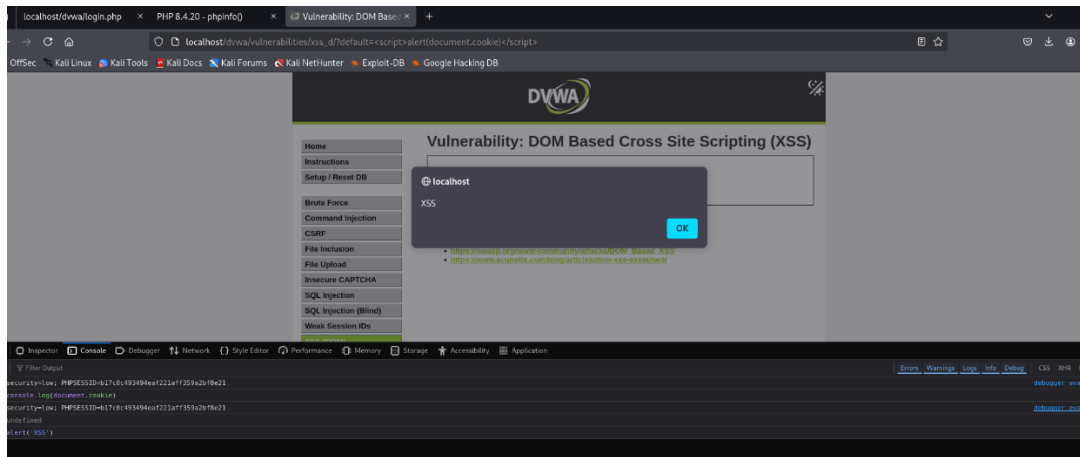
6.1 Cookie Exfiltration

Cookies were successfully exfiltrated using JavaScript redirection to an attacker-controlled server.



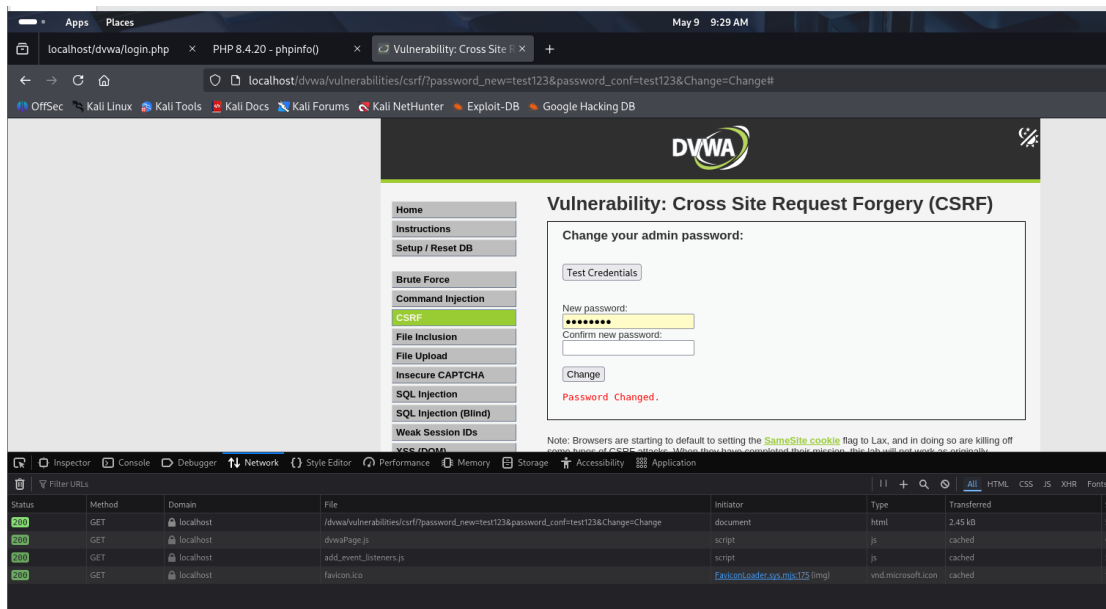
6.2 Keylogging Attack

A client-side keylogger was injected to capture keystrokes and transmit them to an external server.



6.3 Screenshot Capture Test

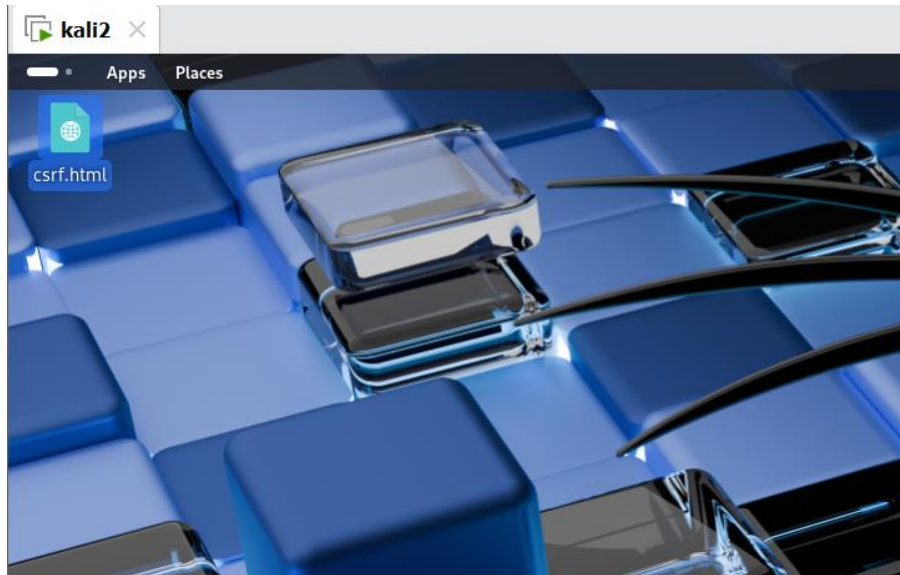
Client-side screenshot capture was simulated using dynamically loaded JavaScript libraries.



7. Cross-Site Request Forgery (CSRF) Evaluation

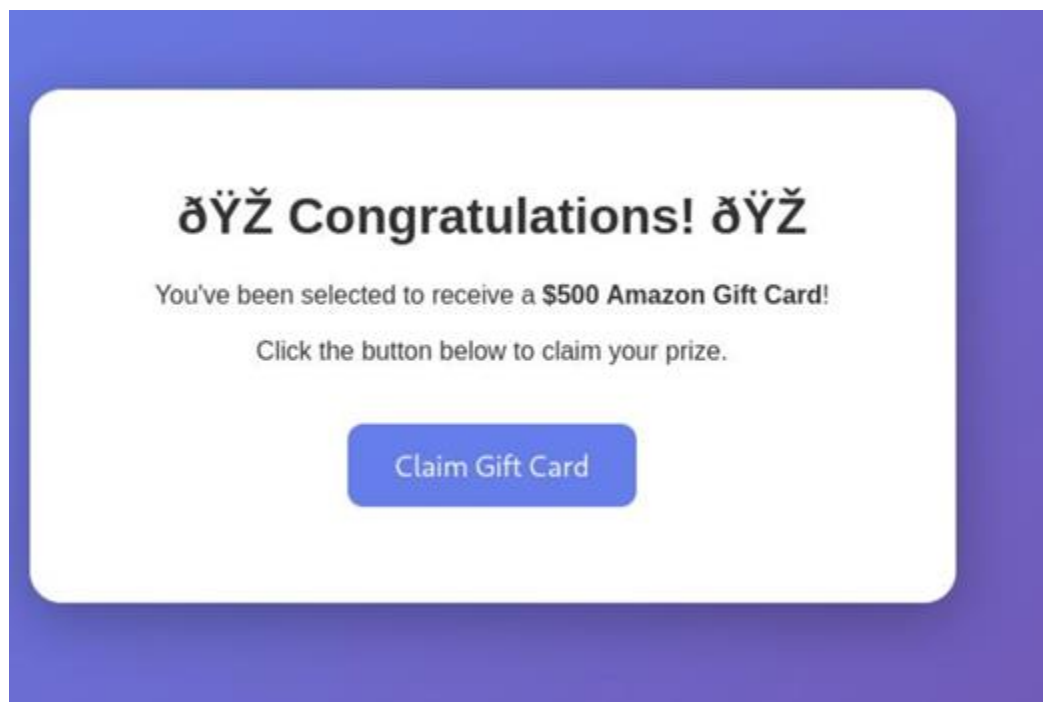
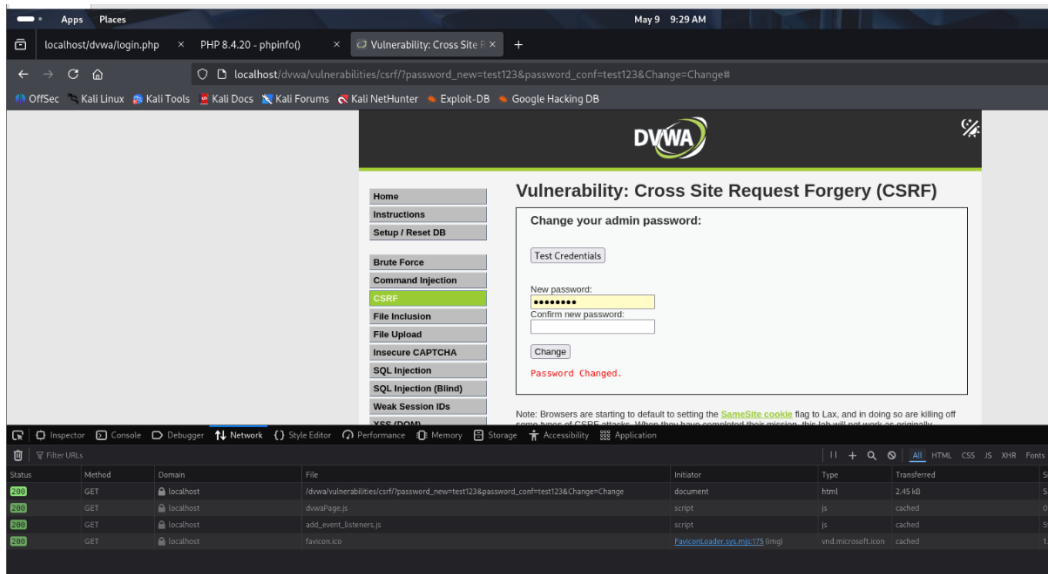
7.1 Attack Scenario

A malicious HTML page was created to silently submit a password change request on behalf of an authenticated victim.



7.2 Attack Validation

- Victim logged into DVWA
- External page triggered hidden POST request
- Password successfully changed





Username
admin

Password
••••••••

Login

Login failed

7.3 Risk Impact

Area	Impact
Account Integrity	Critical
User Awareness	None
Overall Risk	High

8. Security Control Effectiveness Review

8.1 XSS Defense Comparison

Security Level	Result
Low	No protection
Medium	Partial filtering (bypassable)

Security Level	Result
High	Output encoding enforced

8.2 Cookie Flag Review

Flag	Status
HttpOnly	Disabled
Secure	Disabled
SameSite	None

```

$ python3 -m http.server 8081
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
192.168.0.107 - - [23/Apr/2026 10:51:10] "GET /csrf_attack.htm
192.168.0.107 - - [23/Apr/2026 10:51:11] code 404, message Fil
192.168.0.107 - - [23/Apr/2026 10:51:11] "GET /favicon.ico HTT

```

9. Consolidated Vulnerability Matrix

ID	Vulnerability	Severity
V-01	Reflected XSS	Critical
V-02	Stored XSS	Critical
V-03	Cookie Theft	Critical
V-04	CSRF Password Change	High
V-05	Missing HttpOnly Flag	High
V-06	Insecure Cookie Transmission	Medium

10. Mitigation & Hardening Recommendations

Priority Actions

- Enforce output encoding globally
- Enable HttpOnly, Secure, and SameSite cookie flags
- Implement CSRF tokens for all state-changing requests
- Deploy Content Security Policy (CSP)
- Replace blacklist filtering with whitelist validation

11. Appendix – Payload Reference Library

(Payloads used during testing – for educational and defensive validation purposes only)

Final Conclusion

This engagement demonstrated that **client-side vulnerabilities remain a significant attack vector** when defensive controls are misconfigured or incomplete. While higher security levels reduced exploitability, improper cookie configuration and missing defense-in-depth controls continued to expose the application to high-risk attacks.

A layered security approach combining **input validation, output encoding, secure cookies, CSP, and CSRF protections** is essential to effectively mitigate client-side threats.